



Liquidation Improvements

Audit Report

June 03, 2026 Version 1.0

Table of Contents

01 Executive Summary	3
Overview	3
Audit Scope	3
Summary of Findings	4
Findings & Resolutions	4
02 Introduction	6
About Cyba Blockchain Security	6
About The Reviewed Functionality	6
Issues Risk Classification	6
Impact	7
Likelihood	7
Actions required by severity level	7
Informational findings	7
03 Findings	8
Medium Severity Findings	8
[M-01] Integer Floor on total-collaterals-liquid-value Causes repay-allowed > debt, Re- verting Liquidations	8
[M-02] DivisionByZero in socializbad-debt Permanently Blocks Liquidation of Some Bad-Debt Positions	11
Low Severity Findings	14
[L-01] Staking Share Inflation Attack Enables Theft From Subsequent Stakers On New Markets	14
Informational Findings	18
[I-01] Liquidator Contract Minor Optimizations and Unused Variables	18
04 Legal Disclaimer	21

01 | Executive Summary

Granite engaged Cyba Blockchain Security to conduct a security review of the codebase contained within the [core-v1](#) repository.

The engagement was conducted over a period of one day, from 29.05.2026 to 29.05.2026, during which a team of one auditor reviewed the files and commits included in scope.

The audit focused on a manual review of the codebase, with the objective of identifying security vulnerabilities as well as providing codebase enhancement and fortification recommendations. These recommendations were documented as informational findings where applicable.

During the engagement, a total of 3 issues were identified. All identified issues were either resolved by the development team or explicitly acknowledged. In addition, 1 informational finding related to code quality and maintainability was reported, and was resolved by the team.

Overview

Project Name	Liquidation Improvements
Codebase	https://github.com/GraniteProtocol/core-v1
Operating platform	Stacks
Programming language	Clarity
Initial commit	ecf8d1ebc8270d68ad1560dc59ea403df6e2d3bf
Remediation commit	a840837efc85d4057452b6376157aa94a93001b5
Timeline	From 29.05.2026 to 29.05.2026 (1 day)
Audit methodology	Static analysis and manual review
Auditors	Alin Barbatei (ABA)

Audit Scope

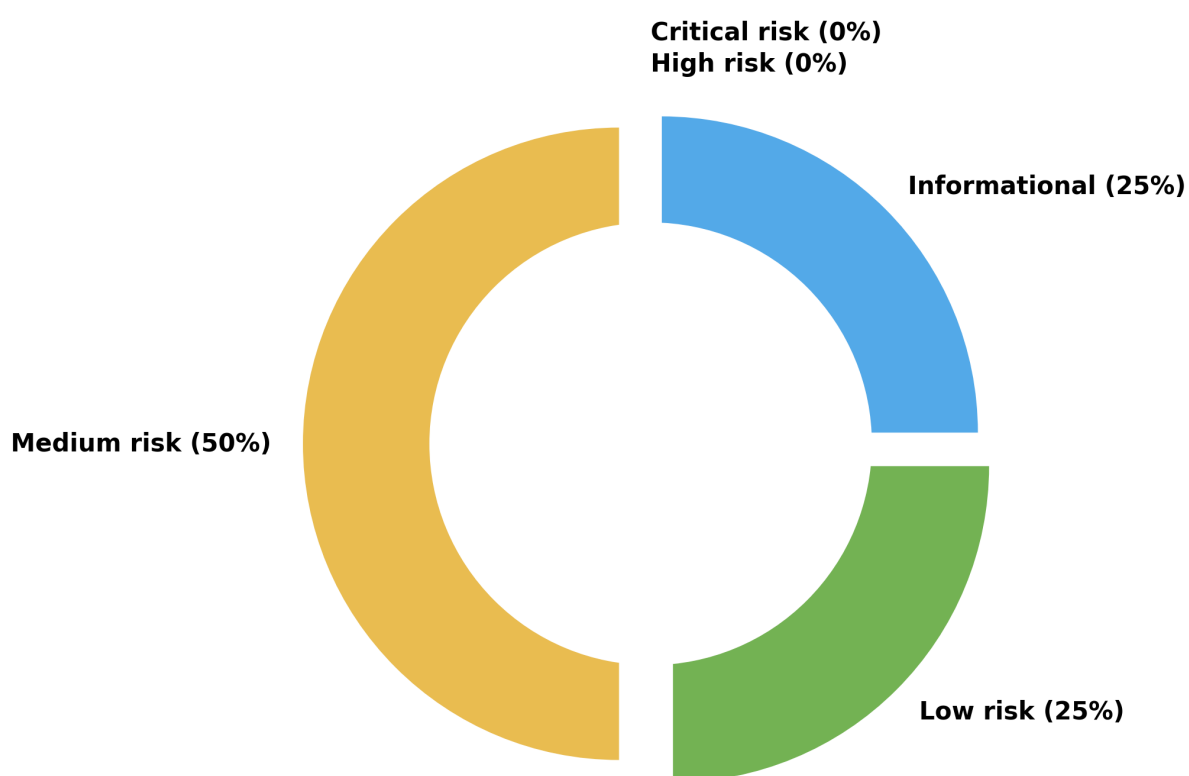
The audit focused on recent changes to the liquidation logic and an extra finding in the staking logic.

Files and folders in scope

- `contracts/liquidator-v1.clar`
 - `contracts/staking-v1.clar`
-

Summary of Findings

Severity	Total Found	Resolved	Partially Resolved	Acknowledged
Critical risk	0	0	0	0
High risk	0	0	0	0
Medium risk	2	2	0	0
Low risk	1	1	0	0
Informational	1	1	0	0



Findings & Resolutions

ID	Title	Severity	Status
M-01	Integer Floor on total-collaterals-liquid-value Causes repay-allowed > debt, Reverting Liquidations	Medium	Resolved

ID	Title	Severity	Status
M-02	DivisionByZero in socializbad-debt Permanently Blocks Liquidation of Some Bad-Debt Positions	Medium	Resolved
L-01	Staking Share Inflation Attack Enables Theft From Subsequent Stakers On New Markets	Low	Resolved
I-01	Liquidator Contract Minor Optimizations and Unused Variables	Informational	Resolved

02 | Introduction

About Cyba Blockchain Security

Cyba Blockchain Security is a security firm founded by [Alin Barbatei \(ABA\)](#), focused on rigorous smart contract and protocol security across EVM and Bitcoin L2 ecosystems, including Stacks (Clarity).

Built on a foundation of deep Web2 and Web3 security experience, Cyba emphasizes high-quality, adversarial reviews and clear, step-by-step guidance throughout every engagement, helping teams not only identify risks, but understand and resolve them with confidence.

For more information visit cybasecurity.io.

About The Reviewed Functionality

Granite is a DeFi lending market that offers overcollateralized loans on [SIP-10](#) tokens managed and operated by immutable smart contracts. Granite was created and is managed by [Trust Machines](#), a leading team of engineers, builders and researchers within the [Stacks Bitcoin L2](#) ecosystem.

This engagement reviewed changes done to the liquidation logic with regards to repay amount denomination. During the review a few other issues were found in adjacent code and included in the findings.

The extra issues revolved around edge cases in socializing bad debt and an impractical, but theoretically possible, staking contract inflation attack.

Issues Risk Classification

The current report contains issues, or findings, that impact the protocol. Depending on the likelihood of the issue appearing and its impact (damage), an issue is in one of five risk categories or severities: *Critical, High, Medium, Low or Informational*.

The following table shows an overview of how likelihood and impact determines the severity of an issue.

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost or a functionality of the protocol is affected.
- **Low** - any kind of unexpected behavior that's not so critical.

Likelihood

- **High** - direct attack vector; the cost is relatively low to the amount of funds that can be lost.
- **Medium** - only conditionally incentivized attack vector, but still relatively likely.
- **Low** - too many or too unlikely assumptions; provides little or no incentive.

Actions required by severity level

- **Critical** - client **must** fix the issue.
- **High** - client **must** fix the issue.
- **Medium** - client **should** fix the issue.
- **Low** - client **could** fix the issue.

Informational findings

Informational findings encompass recommendations to enhance code style, operations alignment with industry best practices, gas optimizations, adherence to documentation, standards, and overall contract design.

Informational findings typically have minimal impact on code functionality or security risk.

Evaluating all vulnerability types, including informational ones, is crucial for a comprehensive security audit to ensure robustness and reliability.

03 | Findings

Medium Severity Findings

[M-01] Integer Floor on total-collaterals-liquid-value Causes repay-allowed > debt, Reverting Liquidations

Severity: *Medium risk* (Resolved) [PoC]

Context: *liquidator-v1.clar:370-383,273-274,452-459 statv1.clar:843 math-v1.clar:5-8*

Description

The latest change to the liquidation logic correctly converts `repay-amount-value` (`USD\VALUE`) to `repay-amount` (`TOKEN\AMOUNT`) using `divide-round-up` at [L273-274](#):

```
(repay-amount-value (get repay-amount liquidation-info))
(repay-amount (contract-call? .math-v1 divide-round-up (* repay-amount-value
  PRICE-SCALING-FACTOR) market-asset-price))
```

Round-up is the correct semantic choice (protocol's favor).

However, the upstream `calculate-repayment-info` formula can produce `repay-allowed > debt` due to integer floor effects, and the round-up amplifies this during USDC depeg. The excess causes an arithmetic underflow in `state-v1` (immutable contract), reverting the liquidation.

`total-collaterals-liquid-value` is computed as `floor(collateral-value * liquidation-ltv / SCALING-FACTOR)` in `iterate-collateral-value-ltv` at [L452-459](#). The floor can discard up to 1 unit. In `calculate-repayment-info` at [L370-383](#):

```
(total-repay-amount (try! (safe-div (* (- debt total-collaterals-liquid-value)
  SCALING-FACTOR) denominator)))
(repay-amount-without-discount (contract-call? .math-v1 divide-round-up (*
  collateral-value SCALING-FACTOR) (+ liquidation-discount SCALING-FACTOR)))
(repay-allowed (if (< total-repay-amount repay-amount-without-discount)
  total-repay-amount repay-amount-without-discount))
```

The continuous-math proof that `repay-allowed <= debt` relies on `total-repay-amount > debt` and `repay-amount-without-discount > debt` being mutually exclusive. The integer floor on `total-collaterals-liquid-value` breaks this, it can be smaller than its continuous value, making both conditions satisfiable simultaneously and producing `repay-allowed = debt + 1`.

For `liquidation-discount = 20%` and `liquidation-ltv = 50%`, any position where `debt ≡ 4 (mod 5)` with `collateral-value = ceil(1.2 * debt)` triggers this. That is approximately 20% of all positions. The excess is always exactly 1 unit and the pattern holds at every scale:

debt	collateral-value	total-collaterals-liquid-value	repay-allowed	repay-allowed - debt
9	11	5	10	+1
99,999,999 (\$1)	119,999,999	59,999,999	100,000,000	+1
99,999,999,999 (\$1K)	119,999,999,999	59,999,999,999	100,000,000,000	+1

When $\text{repay-allowed} = \text{debt} + 1$ flows downstream:

1. $\text{repay-amount} = \text{divide-round-up}((\text{debt} + 1) * \text{SCALING-FACTOR} / \text{market-asset-price})$, at USDC=\$1 this is $\text{debt} + 1$
2. $\text{paid-shares} = \text{floor}((\text{debt} + 1) * \text{total-debt-shares} / \text{open-interest})$, with zero interest ($\text{open-interest} = \text{total-debt-shares}$), this is $\text{debt} + 1$
3. state-v1 L843: $(- (\text{get debt-shares position}) \text{paid-shares}) = (- \text{debt} (\text{debt} + 1)) \rightarrow \text{uint underflow} \rightarrow \text{Clarity runtime abort}$

With any non-zero interest accrual ($\text{open-interest} > \text{total-debt-shares}$), the floor in step 2 absorbs the +1 at USDC=\$1. However, **during a USDC depeg** the price conversion at step 1 amplifies the +1 by $\text{SCALING-FACTOR} / \text{market-asset-price}$, overwhelming the interest absorption at any realistic interest level:

Interest	USDC=\$1.00	USDC=\$0.90	USDC=\$0.50
0%	Underflow	Underflow	Underflow
0.01%	Safe	Underflow	Underflow
1%	Safe	Underflow	Underflow
10%	Safe	Underflow	Underflow

USDC depegged to ~\$0.88 in March 2023. At that level, the amplification factor is ~1.14x, enough to overwhelm interest absorption for typical positions. During any USDC depeg, ~20% of all unhealthy positions become permanently unliquidatable. The M-06 fix was designed to handle USDC depeg, but it breaks liquidation during the exact scenario it targets.

Unliquidatable positions accumulate bad debt that gets socialized to LPs and stakers.

Recommendation

Two caps are needed, in two different domains:

Cap 1, USD-value domain. Cap repay-allowed at debt inside $\text{calculate-repayment-info}$. This prevents the floor-effect excess from the repayment formula:

```
(total-repay-amount (try! (safe-div (* (- debt
    total-collaterals-liquid-value) SCALING-FACTOR)
    denominator)))
```

```

      (repay-amount-without-discount (contract-call? .math-v1
        divide-round-up (* collateral-value SCALING-FACTOR) (+
          liquidation-discount SCALING-FACTOR)))
-    (repay-allowed (if (< total-repay-amount
      repay-amount-without-discount) total-repay-amount
      repay-amount-without-discount))
+    (repay-allowed-row (if (< total-repay-amount
      repay-amount-without-discount) total-repay-amount
      repay-amount-without-discount))
+    (repay-allowed (if (< repay-allowed-row debt)
      repay-allowed-row debt))

```

Cap 2, TOKEN-amount domain. Cap repay-amount at current-debt in execute-liquidation, after the divide-round-up price conversion. This prevents the double-ceiling inflation:

```

      (repay-amount-value (get repay-amount liquidation-info))
-    (repay-amount (contract-call? .math-v1 divide-round-up
      (* repay-amount-value PRICE-SCALING-FACTOR)
      market-asset-price))
+    (repay-amount-row (contract-call? .math-v1
      divide-round-up (* repay-amount-value PRICE-SCALING-FACTOR)
      market-asset-price))
+    (repay-amount (if (< repay-amount-row current-debt)
      repay-amount-row current-debt))

```

Why both are needed:

Cap 1 alone is sufficient at USDC=\$1.00 (the price conversion is a no-op), but NOT during a depeg. The round-trip $\text{ceil}(\text{token} \rightarrow \text{USD}) \rightarrow \text{ceil}(\text{USD} \rightarrow \text{token})$ inflates by +1 in the token domain even when $\text{repay-allowed} = \text{debt}$ in the USD domain:

```

current-debt (tokens) = 101, market-asset-price = 900000000 ($0.90)

debt (USD)           = ceil(101 × 0.9)           = ceil(90.9)           = 91
repay-allowed        = min(formula, 91)          = 91                   (cap 1 applied)
repay-amount         = ceil(91 × 1e8 / 9e7)       = ceil(101.111)       = 102 ←exceeds
  current-debt!
paid-shares          = floor(102 × 101/101)        = 102                  (zero interest)
user-debt-shares     = 101
state-v1 L843: (- 101 102) → UNDERFLOW

```

This happens for any current-debt where $\text{current-debt} \times \text{market-asset-price}$ is not divisible by SCALING-FACTOR, which is most positions during any depeg.

Cap 2 prevents this: $\text{repay-amount} = \min(102, 101) = 101$. Then $\text{paid-shares} = \text{floor}(101 \times 101 / 101) = 101 = \text{user-debt-shares}$. No underflow. This is mathematically guaranteed because $\text{current-debt} = \text{ceil}(\text{user-debt-shares} \times \text{open-interest} / \text{total-debt-shares})$, and $\text{floor}(\text{current-debt} \times \text{total-debt-shares} / \text{open-interest}) \leq \text{user-debt-shares}$ always holds by the ceil/floor relationship.

Cap 1 alone would miss the depeg case. Cap 2 alone would miss the floor-effect repay-allowed > debt (which also inflates collateral-to-give in the USD domain, the liquidator receives more collateral than they should). Together they close both gaps.

A third alternative, capping only paid-shares at user-debt-shares, prevents the underflow at state-v1 L843 but does **not** fix the problem. The uncapped repay-amount still flows into calculate-interest-portions, producing an inflated principal-part that gets added to lp-part and passed to state-v1 L852: $(- \text{lp-open-interest } \text{lp-part})$. When the user is the sole or dominant borrower, this underflows at L852 instead of L843.

Resolution

Resolved, the recommended fix was implemented in [PR#66](#).

[M-02] DivisionByZero in socialize-bad-debt Permanently Blocks Liquidation of Some Bad-Debt Positions

Severity: *Medium risk* (Resolved) [PoC]

Context: [math-v1.clar:5-8,28-35](#) [liquidator-v1.clar:512-567,489-496](#)

Description

calculate-interest-portions at [math-v1.clar L28-35](#) splits a repayment into principal and interest portions by dividing by current-debt:

```
(define-read-only (calculate-interest-portions (current-debt uint) (borrowed-amount
uint) (repay-amount uint))
  (let (
    (interest-accrued (- current-debt borrowed-amount))
    (interest-part (divide-round-up (* interest-accrued repay-amount)
current-debt))
    (principal-part (- repay-amount interest-part))
  )
    { principal-part: principal-part, interest-part: interest-part }
  ))
```

divide-round-up at [L5-8](#) evaluates $(\text{mod } \text{numerator } \text{denominator})$ before any guard. When $\text{current-debt} = 0$, this becomes $(\text{mod } 0 \ 0)$, a Clarity runtime panic (DivisionByZero). Unlike safe-div (which returns 0 on zero denominator), calculate-interest-portions has no zero-denominator guard.

The socialize-bad-debt function at [liquidator-v1.clar L512-567](#) triggers this. After a liquidation fully pays off a bad-debt position, socialize-bad-debt re-reads the user's position from state. But the liquidation state update has already zeroed debt-shares and borrowed-amount. At [L523](#), $\text{remaining-debt} = \text{convert-to-debt-assets}(\{\text{open-interest: } 0, \text{total-debt-shares: } 0\}, 0, \text{true}) = 0$. At [L529](#), $\text{user-borrowed-amount} = 0$. The guard at [L535](#) passes because $(\geq 0 \ 0)$ is true, and calculate-interest-portions(0, 0, 0) is called $\rightarrow$ DivisionByZero $\rightarrow$ the entire transaction aborts, leaving the position permanently stuck.

For this to trigger, three conditions must hold simultaneously:

1. **Bad debt:** `collateral-value < debt + floor(collateral-value * liquidation-discount / SCALING-FACTOR)`. The `is-bad-debt` check at [L489-496](#) includes the liquidation premium in the threshold, so positions where `collateral-value > debt` can still be flagged as bad debt.
2. **Full payoff:** `repay-allowed = debt` exactly.
3. **All collateral seized:** `collateral-to-give = collateral-value`.

Concrete counterexample: `debt = 14`, `collateral-value = 16`, `liquidation-discount = 20%`, `liquidation-ltv = 50%`.

- Bad-debt check: `collateral-reward = floor(16 * 20000000 / 1e8) = 3`. Is `16 < 14 + 3 = 17`? Yes → bad debt.
- Liquidation formula: `total-collaterals-liquid-value = floor(16 * 0.5) = 8`. `denominator = 4e7`. `total-repay-amount = floor((14-8) * 1e8 / 4e7) = 15`. `repay-amount-without-discount = ceil(16 * 1e8 / 1.2e8) = 14`. `repay-allowed = min(15, 14) = 14 = debt` → full payoff.
- Collateral seizure: `floor(14 * 1.2) = 16 = collateral-value` → all seized.
- State update succeeds (zeroes debt-shares and borrowed-amount).
- Post-liquidation health check passes (`debt = 0` → `MINIMUM_HEALTH_RATIO`).
- `socialize-bad-debt` calls `calculate-interest-portions(0, 0, 0)` → (`mod 0 0`) → **DivisionByZero** → transaction reverts.

The position is now permanently unliquidatable: unhealthy, flagged as bad debt, but every liquidation attempt reverts identically. It accumulates more bad debt over time with no resolution path.

Recommendation

Add a `remaining-debt = 0` early exit in `socialize-bad-debt` at the existing `total-collateral-value` check ([L526](#)). This prevents the function from entering the socialization `let` block when there is nothing to socialize. Example implementation:

```
;; liquidator-v1.clar, socialize-bad-debt (L526)
-   (if (> total-collateral-value u0)
+   (if (or (> total-collateral-value u0) (is-eq
    remaining-debt u0))
        SUCCESS
        (let (
```

`remaining-debt` is already bound at [L523](#) before this check. When `remaining-debt = 0` (full payoff zeroed debt-shares), the early exit avoids `calculate-interest-portions(0, 0, 0)` entirely. The existing socialization logic inside the `let` block requires no changes.

The other two `calculate-interest-portions` call sites are already safe:

- **execute-liquidation (L281):** unreachable with `current-debt = 0`. The pre-liquidation check `check-account-unhealthy (L435)` uses `strict <: health < MINIMUM_HEALTH_RATIO`. When `debt = 0`, `health = MINIMUM_HEALTH_RATIO`, and `1e8 < 1e8` is false, the position is rejected as healthy before `execute-liquidation` runs. Additionally, L281 is evaluated in the `let` bindings before the state update, so `current-debt` always reflects the original (non-zero) pre-liquidation value.

- **borrower-v1.clar (L91)**: guarded by `(asserts! (> current-debt u0) ERR-NO-DEBT)` at [L89](#).

When the guard activates (`remaining-debt = 0` after a full payoff), the socialization is correctly skipped. The liquidation already fully covered the debt. The `is-bad-debt` flag was set by the premium-inclusive threshold (`collateral-value < debt + reward`), but the position was actually solvent (`collateral-value > debt`), so there is nothing to socialize.

Resolution

Resolved, the recommended fix was implemented in [PR#67](#).

Low Severity Findings

[L-01] Staking Share Inflation Attack Enables Theft From Subsequent Stakers On New Markets

Severity: *Low risk* (Resolved) [PoC]

Context: *staking-v1.clar:97-106,167-186,189-209*

Description

The staking module uses a share-based vault model (staked-lp-token shares backed by total-lp-tokens-staked LP tokens) but has no protection against share price inflation. An attacker who is the first staker can inflate the share price through stake/unstake cycles at zero net cost, then steal ~25% of any subsequent staker's deposit via rounding.

In convert-to-staked-lp-tokens (L97-106), when ft-get-supply staked-lp-token = 0, the first staker gets 1:1 shares with no minimum and no dead shares:

```
(if (is-eq total-staked-lp-tokens u0)
  lp-tokens ;; 1:1 for first staker, no dead shares
  (contract-call? .math-v1 divide round-up (* total-staked-lp-tokens lp-tokens)
    total-lp-tokens)
)
```

The stake() function (L167-186) only validates (> lp-tokens u0) at L174 but never checks staked-lp-tokens-to-mint > 0. When the share price is inflated enough that convert-to-staked-lp-tokens returns 0, ft-mint? staked-lp-token u0 reverts with (err u1).

Attack path:

1. Attacker is the first staker. Stakes N LP tokens, receives N staked-LP shares (1:1).
2. Some interest accrues (even 1 unit is enough). Pool: $(N + 1) \text{ LP} / N \text{ shares}$.
3. Attacker unstakes $(N - 1)$ shares, waits the 100-block finalization period, finalizes. Pool: $\sim 2 \text{ LP} / 1 \text{ share}$.
4. Attacker repeats stake/unstake cycles to inflate. Each cycle:
 - Stake $(2A - 1)$ LP tokens $\rightarrow \text{floor}(1 * (2A-1) / A) = 1$ share minted. Pool: $(3A - 1) \text{ LP} / 2 \text{ shares}$.
 - Unstake 1 share $\rightarrow \text{floor}(1 * (3A-1) / 2) \approx 3A/2$ LP returned. Pool: $\sim 3A/2 \text{ LP} / 1 \text{ share}$.
 - Share price grows $\sim 1.5x$ per cycle. Attacker's total wealth (cash + share value) is preserved, the inflation costs nothing beyond tx fees.
5. **Theft.** Attacker inflates to $A \approx D/2$ (where D is the victim's expected deposit). Victim stakes D LP tokens $\rightarrow \text{floor}(1 * D / A) = 1$ share (rounding loss). Pool: $(A + D) \text{ LP} / 2 \text{ shares}$. Attacker unstakes for $\text{floor}((A + D) / 2) \approx 3D/4$. Of that, D/2 is the attacker's own capital recovered, and D/4 is pure profit stolen from the victim. Victim's remaining share is worth $\sim 3D/4$, they lost $\sim 25\%$ of their deposit.

6. **DOS (secondary).** Victims staking below the share price get 0 shares $\rightarrow$ `ft-mint?` (`u0`) reverts with (`err u1`) $\rightarrow$ transaction fails entirely (LP tokens are not transferred because the revert rolls back the full tx).

The share price grows geometrically: $2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 9 \rightarrow 13 \rightarrow 19 \rightarrow 28 \rightarrow 42 \rightarrow 63 \rightarrow 94 \rightarrow \dots$. After 25 cycles, the share price exceeds 50,000, blocking any stake below ~\$0.05 (at 6 decimal precision).

Feasibility: Each cycle requires a 100-block finalization period (`withdrawal-finalization-period`, [L26](#)). On Stacks mainnet [average per block](#) for the last 30 days is 7.74s, so each cycle takes ~13 minutes.

Some examples:

- To reach a share price that blocks deposits under \$1 USDC (~33 cycles) requires ~7.1 hours of stake/unstake cycling
- Blocking deposits under \$1,000 USDC (~50 cycles) requires ~10.8 hours.

The attacker's capital is fully recovered each cycle (no net cost beyond tx fees). Interest accrual via `increase-lp-staked-balance` ([L189-209](#)) accelerates the inflation when staking rewards are active but is not required for the attack.

Note: the direct-donation inflation vector (transferring LP tokens directly to the staking contract) is not viable here because `convert-to-staked-lp-tokens` uses the internal `total-lp-tokens-staked` variable ([L100](#)), not the contract's LP balance. The `reconcile-lp-token-balance` function ([L250](#)) that syncs internal accounting to the balance is governance-only. This finding uses the stake/unstake cycle vector instead.

Recommendation

Add an `initialize()` function with a minimum initial stake burned to an unspendable address (dead shares), preventing the share supply from ever collapsing to a manipulable level.

Example implementation:

```
(define-constant ERR-INTEREST-PARAMS (err u60008))
(define-constant ERR-STAKING-DISABLED (err u60009))
(define-constant ERR-STAKING-WIPED-OUT (err u60010))
+(define-constant ERR-ALREADY-INITIALIZED (err u60011))
+(define-constant ERR-NOT-INITIALIZED (err u60012))

;; Constants
(define-constant SUCCESS (ok true))
+(define-constant MINIMUM_INITIAL_STAKE u1000)
+(define-constant NULL_ADDRESS (unwrap-panic
  (principal-construct? (if is-in-mainnet 0x16 0x1a)
    0x0000000000000000000000000000000000000000000000000000000000000000)))

;; lp-token
(define-constant token-prefix "gusdc")
```

```

-29,6 +33,7
(define-map user-withdrawals { user: principal, index: uint } {
  withdrawal-shares: uint, finalization-at: uint })
(define-data-var total-lp-tokens-staked uint u0)
(define-data-var staking-wiped-out bool false)
+(define-data-var initialized bool false)

;; governance
(define-public (update-withdrawal-finalization-period
  (new-value uint))
-163,6 +168,33
  (var-get staking-wiped-out)
)

+(define-public (initialize)
+ (let (
+   (total-staked (ft-get-supply staked-lp-token))
+   (dust-burned (is-eq total-staked u0))
+   (initial-balance (ft-get-balance staked-lp-token
contract-caller))
+ )
+ (asserts! (not (var-get initialized))
  ERR-ALREADY-INITIALIZED)
+ (var-set initialized true)
+ (try! (if dust-burned
+   (let (
+     (status (try! (stake MINIMUM_INITIAL_STAKE)))
+     (stake-balance-increase (- (ft-get-balance
staked-lp-token contract-caller) initial-balance))
+   )
+   (transfer stake-balance-increase contract-caller
  NULL_ADDRESS none)
+   )
+   SUCCESS
+ ))
+ (print {
+   action: "initialized",
+   user: contract-caller,
+   total-staked: total-staked,
+   dust-burned: dust-burned,
+ })
+ SUCCESS
+ )
+)
+

```

```
;; Public functions
(define-public (stake (lp-tokens uint))
  (begin
    -172,6 +204,7
    (staked-lp-tokens-to-mint (convert-to-staked-lp-tokens
      lp-tokens false))
    )
    (asserts! (> lp-tokens u0) ERR-ZERO-LP-TOKEN-STAKE)
+   (asserts! (var-get initialized) ERR-NOT-INITIALIZED)
    (try! (contract-call? .state-v1 transfer lp-tokens
      contract-caller (as-contract contract-caller) none))
    (try! (ft-mint? staked-lp-token staked-lp-tokens-to-mint
      contract-caller))
    (var-set total-lp-tokens-staked (+ (var-get
      total-lp-tokens-staked) lp-tokens))
```

`initialize()` reuses `stake()` to handle LP transfer, share minting, and counter updates, then transfers the resulting shares from the caller to `NULL_ADDRESS`. This avoids duplicating the stake logic. The `initialized` flag is set before the internal `stake()` call so the gate check passes. On upgrades where shares already exist, the dust burn is skipped.

The dead shares ensure `ft-get-supply staked-lp-token` is always > 0 , preventing the share supply from collapsing to 1. The `NULL_ADDRESS` holds shares that can never be redeemed, anchoring the share price.

Note, on already deployed markets, where this issue is not relevant (at least not until all stakers choose to unstake) governance/devs can just manually transfer a dust amount of staked LP tokens to the null address to ensure this roundabout sub-case cannot reoccur.

Resolution

Resolved, the recommended fix was implemented in [PR#68](#).

Informational Findings

[I-01] Liquidator Contract Minor Optimizations and Unused Variables

Severity: *Informational* (Resolved)

Context: *liquidator-v1.clar:261,275-276,129,268,358*

Description

Three minor code quality improvements in `liquidator-v1.clar`:

1. **Redundant market-asset-price oracle call.** `execute-liquidation` fetches `market-asset-price` at [L275](#), but `get-liquidation-info` (called at [L261](#) with `none` for `maybe-market-asset-price`) also fetches it internally via `account-health`. Moving the fetch earlier and passing it as `(some market-asset-price)` avoids the redundant oracle read and makes the price available for the downstream health check without re-fetching.
2. **SCALING-FACTOR used where PRICE-SCALING-FACTOR is semantically correct.** At [L276](#), the `divide-round-up` conversion from `USD-value` to `token-amount` uses `SCALING-FACTOR`. Both `SCALING-FACTOR` and `PRICE-SCALING-FACTOR` resolve to the same value (`u100000000`), but the operation is a price-domain conversion, so `PRICE-SCALING-FACTOR` is the semantically correct constant. This was previously flagged as [QA-07] `Scaling Factor Ambiguities` in the Clarity Alliance audit, which recommended using `PRICE-SCALING-FACTOR` to distinguish price-scaling operations from ratio-scaling operations.
3. **Unused variables.** Three `letbound` variables are extracted but never referenced:
 - [L129](#): `collateral-price` in the `liquidate` read-only wrapper
 - [L268](#): `collateral-price` in `execute-liquidation`
 - [L358](#): `collateral` in `fold-execute-liquidation`

Recommendation

Apply the following optimizations. Example implementation:

```
;; execute-liquidation: move market-asset-price fetch earlier,
   pass to get-liquidation-info, use PRICE-SCALING-FACTOR
-258,7 +258,8
      (user-position-data (contract-call? .state-v1
        get-borrow-repay-params user))
      (position-for-block-check (unwrap! (get user-position
        user-position-data) ERR-NO-POSITION))
      ;; get liquidation info for the user
-      (liquidation-res (try! (get-liquidation-info user
+      collateral liquidator-repay-amount none none none none)))
+      (market-asset-price (unwrap! (contract-call?
        .pyth-adapter-v1 read-price .mock-usdc)
        ERR-MISSING-MARKET-PRICE)))
```

```

+      (liquidation-res (try! (get-liquidation-info user
collateral liquidator-repay-amount (some market-asset-price)
none none none)))
      (liquidation-info (get liquidation-info
liquidation-res))
      (current-debt (get current-debt liquidation-res))
      (user-borrowed-amount (get user-borrowed-amount
liquidation-res))
-272,8 +273,7
      ;; get collateral to give to liquidator
      (collateral-to-give (get collateral-to-give
liquidation-info))
      (repay-amount-value (get repay-amount liquidation-info))
-      (market-asset-price (unwrap! (contract-call?
.pyth-adapter-v1 read-price .mock-usdc)
ERR-MISSING-MARKET-PRICE))
-      (repay-amount (contract-call? .math-v1 divide-round-up
(* repay-amount-value SCALING-FACTOR) market-asset-price))
+      (repay-amount (contract-call? .math-v1 divide-round-up
(* repay-amount-value PRICE-SCALING-FACTOR)
market-asset-price))
      ;; convert repay amount to debt shares
      (debt-params (contract-call? .state-v1 get-debt-params))
      (paid-shares (contract-call? .math-v1
convert-to-debt-shares debt-params repay-amount
false))

```

Remove unused variables:

```

;; liquidate read-only wrapper: remove unused collateral-price
-125,8 +125,7
      maybe-total-liquid-ltv
      maybe-collateral-value
      maybe-collateral-price
-      )))
-      (collateral-price (get collateral-price liquidation-info)))
+      )))
      (try! (ensure-non-zero-repay-amount
liquidator-repay-amount))
      (ok (get liquidation-info liquidation-info)))

;; execute-liquidation: remove unused collateral-price
-267,7 +266,6
      (bad-debt (get bad-debt liquidation-res))
-      (collateral-price (get collateral-price
liquidation-res))

```

```
;; get open interest

;; fold-execute-liquidation: remove unused collateral
-356,7 +354,6
))) (result {collateral: <token-trait>, res: (response bool
      uint}}))
  (let (
-    (collateral (get collateral result))
      (prev-result (get res result))
  )
```

Resolution

Resolved, the recommended fix was implemented in [PR#66](#).

04 | Legal Disclaimer

This report is not, nor should be considered, an endorsement or disapproval of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any product or asset created by any team or project that contracts the consultant to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. The consultant's position is that each company and individual are responsible for their own due diligence and continuous security. The consultant's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology that is analyzed.

The assessment services provided by the consultant is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. Furthermore, because a single assessment can never be considered comprehensive, multiple independent assessments paired with a bug bounty program are always recommend.

For each finding, the consultant provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved, but they may not be tested or functional code. These recommendations are not exhaustive, and the clients are encouraged to consider them as a starting point for further discussion.

The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties. Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, the consultant does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

The consultant retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. The consultant is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. The consultant is furthermore allowed to claim bug bounties from third-parties while doing so.